Question 1

a) The Turing test is a test that is supposed to measure how "human" a machine is. If the machine pass the test, the test person is not able to distinguish between a person and the machine. If the test human can distinguish it, it fails the test. There is the machine, and intermediator, and a "normal human". By having questions, and these questions answered one should be able to make a judgement which of the participants is a machine. The "normal human" is the guesser. If the normal human cannot distinguish the responses from a machine, the machine passes the test.

b) The chatbots of many online shopping sites seems like they are meant as a support tool for customers, but they are basically useless. It is not helpful, because they tend to be too dumb. It seems like they are only capable of looking for certain keywords in a prompt and just redirects a user to the sales pages of the topic. The reason behind the failure might be that it maybe is not meant as a support tool at all, but as a jump rate hindrance so that the user stays on the Elkjøp page for longer, rather than going to the competitor. Also, they mostly seem like rule-based chatbots with simple if/else-tests rather than any NLU. A solution would be to implement a more advanced model, much like an LLM if the intent really were to help customers with problems of the products they bought. Sparebank1's (Norwegian bank) model does a much better job of this and understands human language better, it is based on Copilot. They are also planning on implementing this in their own workflow.

Question 2

1) I would select every column but employee number as a feature as I think all of them but employee number might have an impact. Initially one might think that a lower employee number might mean more job involvement and more "loyal to the establishment", but we see from the table this is not the case. For example, a single young person with a low performance rating might be much more likely to get a new job rather than someone who is more established (higher age, married, etc). So the features are everything but employee number. The target value is job satisfaction. If you are satisfied with your job, you are less likely to leave.

2) Feature selection is the selection of features in which you build your model upon. What properties do you think might influence the output. For example, if you want to measure battery life of a laptop, and you have some domain knowledge, you might consider features such as screen size, processor performance (GHz, number of cores), battery cell size and so on. These properties are in AI known as features. Feature engineering is when you create new ones from the initial ones, that can explain more of your model, this is often a result of random forest, where initial features are combined into one, while still having the initial ones as single features. In this case, a likely feature engineered column would be

"JOB_INVOLVEMENT_SATISFACTION".

3) The job status column is a categorical feature that needs to be transformed. One method to transform this feature could be one hot encoding, where you create n new columns (as many as there are different job statuses), and every one but the one that is true gets a zero. The one that is true (hot), gets a 1. Others are gender and marital status.

4) Distance from home needs to be scaled. And perhaps age. Environment satisfaction, job involvement, job satisfaction and performance rating seems to be in similar range, so we can leave those. One method could be (min, max) where you set a minimum and a maximum number and the computer scales the values accordingly. Scaling is important for the model, so that it does not weigh one column more because the numbers itself are bigger. Or weighs less because they are smaller. A very typical blunder here could be age (18-67) and salary, typically six-seven digits in kroner. This unscaled would put way more weight to the salary than the age, even though the age might impact the likeliness of staying with the company just as much as salary.

Question 3

a) A confusion matrix is an overview of four types of "results" from a total of guesses. Consider a malware detection software. If it discovers malware, and the malware is real, it is known as a true positive and we consider the virus being real, and the detection as a good one. But it can also say it has a found a virus (positive), while it has not actually found it (false). When it says it has found a virus, but there isn't one, this is known as a false positive. Then there is true negative, and false negative. True negative is when it says it did not found a virus, and there is no virus. Then a false negative would be it says it did not find a virus, while the virus is there. A confusion matrix shows all these four cases in a quadrant with a number in each. If the confusion matrix showed 1.0 in True Positive quadrant, and 0.0 in the others, it would be considered a perfect virus detection software, because it catched every virus which existed, and never "lied".

b)

1. The KNN algorithm works by measuring the Euclidean distance to it's nearest neighbors, and ranks how near they are by the result of the distance ordered. The K decided how many clusters the calculation will use. For example, we know that if a person checks in at location x, they are likely to check in at other points in close proximity to location x, and less likely to check in at locations farther away. The K here could then be three persons with three different addresses, checking in to three different clusters of places

2. It evaluates the accuracy by segregating the data into two sets, training and test. Then the model tries to guess guesses from the test set (20% of data) based on the data from the training set (80% of data).

Question 4

a) Supervised learning is training on labeled data in order to find out a predestined question or calculation, while unsupervised learning is training on unlabeled data while not necessarily having a question to answer, where you find hidden patterns and the like. An example of supervised learning could be making a computer find the fastest paths in a car racing game by using the path data from previous players, while unsupervised lets the computer play the game without any predestined goal. We know from Pokemon machine learning on youtube that unsupervised AI tends to let the machine navigate the main character in the game to go along the edges of the map for some reason, this is an example of a hidden pattern.

b)

1. I would select age, gender, annual spending, purchase frequency and preferred product category. I do not think city would have a big impact in this regard. Let's say the features are in a .csv file. I would .read_csv and import the columns into Python. and then would one-hot encode categorical values, and (min, max) ranges to 0-1 in regards to age, annual spending and purchase frequency. Then I would split the data into test and training data and do the K-means algorithm as I do not know how many clusters there are, which K-means are able to do itself.

2. By examining the clusters, one could specialize the advertising to the clusters to get a better conversion rate. You want to convert the attention from the advertising to a sale. For example, there is no point in advertising an expensive train set to a customer with low annual spending and purchase frequency, maybe it would be better to not advertise to this group at all since there is so little chance in converting to a sale. From those features we can also extrapolate that the disposable income probably is low. From the table we can see that the customers with preferred product category = Electronics spend the most, and the most frequently, so from that we can extrapolate that they would be positive on both x and y axis and form a K cluster there, so that is the most clear and obvious group to focus on in the advertisement, since by extrapolating they are likely to have a bigger disposable income. So these are two clear examples of how to utilize the model effectively (avoid that K group, focus on that K group while advertising to get the most out of it).

Question 5

a) We can skip the segmentation step as it is only one "document" in the corpus, so the first step would be tokenization, to segment the document into tokens. Then we would cut out parts of the words to the base, for example "processing" would be made into "proces" (stemming). If one follows up with or just does lemmatization is depending on the goal, but lemmatization is also a step in NLP. The difference is that you don't just cut the word, but replaces it with the base dictionary word. After that, you vectorize the words with either bag of words, TF-IDF or word2vec depending on your goals.

b)

1. The sentiment labels that will be created will be "Negative" (1-2), "Neutral" (3) and "Positive" (4-5).

2. To transform the text into numerical vectors with weighting, one can use the TF-IDF vectorizer, since there are many documents in this corpus. Use the word2vec algorithm to build the NLP model, because word2vec is able to understand the meaning behind the text as well, instead of just counting instances and density and weight of words

Question 6

a) Image features are features upon which an image is built. These features can be structure features (circle, square), edge features, color features, texture features, gray level features, noise features. By combining these and values from 0 to 255 in a grid one can picture anything. For example, a color is not "a" color in computer images. It is three channels of red, green and blue combined in one picture element (pixel). If one is to show red in one pixel, the computer will give a signal strength of 255 in the R channel and 0 in the G and B channel. This gives the output of as much red as the computer can give. If one gave 1 R, 0 G and 0 B, it would still technically be red, but a human would interpret is as black. It is important to note that in an LCD display these channels are "on top of each other", while on an old CRT they are at the side of each other very close and there are some qualitative differences, but the RGB mostly works the same, it just looks a little bit different, you can actually see each R and G and B diode on a CRT screen with your eye. For a LCD/OLED etc+ screen you would need a telescope or something.

b)

1. The most clear issue here is the imbalance of amount of pictures. There should be about the same amount of pictures, but now there is five times as many cat pictures. This is a weighting problem. This is probably out of the scope, but those images might not be approved for this use case in regards to copyright and such.

2. You could import the dataset from scikit-learn and get more from the built-in sample library. You specify how many as an argument in the import statement. You could also alter the test size from 0.2 to a smaller number to increase the number of training samples (by losing test samples).

Question 7

a)

1. The forward propagation technique use by taking the input layer as variables to "see" what the next step is likely to be based on what it already knows. So for example, unlike a circle or a triangle, it will learn fast what a square is because it it the same values each pixel propagation. A circle is just many squares (in computers), so it needs to understand the pixel placement representation of a circle as grid values, and see how it increases and decreases by position to learn that it is a circle. It starts with empty or one pixel (leftmost outer square of circle), then sees it go both right, and splitting up and down on the y-axis.

2. Backpropagation is the "checking of the homework", by using the results of the forward propagation, it checks that the same is true with unknown factors. For example, if we say forward propagation is 2 + 2 = x, then backwards propagation is 2 + x = 4, where it finds x. The purpose is to check whether the calculations are right to heighten the self-certainty factor and not just accuracy.

Question 8

a) Input layer, outer layer, activation layer, hidden layer. The input layer is the layer which contains the image information in regards to file size, dimensions, color and so on. The outer layer is the presentation layer, the activation layer is what makes it viewable by a human, and the hidden layer contains metadata which is presented beside the picture.

b)

1. Epoch is a complete run through the dataset with training and test data. To better the model it is normal to run through the dataset several times. The amount of times are epochs. Accuracy is a simple measurement of how many times the model guessed right. It is important to note that accuracy is a different metric of how sure the model is, it if guessed what was a cat and what was a dog and it guessed right 100% of the times, it would have 100% accuracy by luck. Loss is how much instances it loses. If the model have high loss, it is very unsure and likely have low accuracy.

2. The specific evidence of overfitting is the graph sinking quickly in the accuracy diagram and the spike in the loss diagram. One approach to mitigate it would be to run fewer epochs, or

make the input dataset smaller. It might simply be too many pictures.

Question 9

a) Rule-based chatbots are simpler and typically keyword-based. It does not "understand" the messages from the human the same way an AI-powered chatbot does. A rule-based chatbots typically just sees a word, and then acts based upon that word regardless of context. An AI-powered chatbot might understand, for example, that you do not necessarily want to buy a phone model just because you mention that phone model, but maybe that phone model + problem is indicative of you wanting a replacement, or making a guarantee case issue and so on. An example of a rule-based chatbots are the irritating useless ones on e-commerce sites, which is unproductive and dumb as rocks where you are better of using a traditional UI with text and buttons and images. An example of an AI-powered chatbot could be Claude, it is very good for explaining concepts within AI and coding, it is even helpful for generating code.

b) AI-powered chatbots can benefit programmers specifically by aiding with code. Claude is very good for this. Both explaining it, creating it, modifying it and explain what it does good, bad, and how it could be better. It can benefit by generating boilerplate code and reduce time waste. It can also send you in the wrong direction and make code that technically works and solves your problem but send 1000 requests where you need 1 and you won't notice the difference. You can get a bigger context window by using add-ons directly in your IDE rather than "externally" in a browser window, and having it write code "agentically" but this does not run without risks. One bad prompt and your production database could be wiped. It can also help students of Information Technology and Information Systems by over time understanding how you speak and talk, and then put itself on the same level. In real life, people speaking differently can hinder understanding and be a drag on the speed of learning. By utilizing AI alone in studying one can "equalize" each other and then have more productive meetings when everyone is on the same page, for example calling <p> "the p tag" instead of "that crocodile symbol with the p for text" which makes no sense (1st semester).

Question 10

a) I think it should. "Human-like" is very unspecific. A dog could also be human-like, and few people have problems with this, and many humans use dogs as social companions even if they can't have deep conversations, they still can have genuine social connections. We give our daughters dolls to play with and boys action figures, and they learn skills from this play, and none of them think they're real, they know the play is a simulation. People who make up worlds and people are called writers and authors and we don't see them as parasocial hermits in dire need of socialization, but successful people (well, the ones we know about).

AI is just another medium and helping aid, and the problem is not the AI itself, it is how we use it, and sometimes how it is designed to "hook" a user to continued use. We know we shouldn't drive everywhere and that we should walk so and so much. We know to understand something we need to repeat it many times, it is not enough to read. AI is just another tool that we need to learn to use.

And just as we learn to use social media and the smartphone, we need to understand that the phone itself is not the danger, but the people behind it who designs it to be addictive through the use of social validation feedback. The same way we need to understand that just because AI says it agrees with us, it is not necessarily right. And also, that it can refer to statistics as the truth and the final answer to everything, without acknowledging that statistics are just a try at representing reality, filtered through a lens. There are many steps from an instance of "a statistic" happening until it is a number in a dataset. So yes, I think one should embrace it, but it is important to have some "information campaign" of what AI is and is not. It seems like most people understand AI as an oracle rather than a owerpowered mathematical word vectorizer with internet browsing capabilities.

b) I do think that ultimately right now AI is making people dumber because it is not used right and people have not learned to use it right. Just like the advent of home-TV made people lazy and lowered their self-reasoning capabilities based on one's experienced and learned life, AI is in a way doing the same because of the wrong way most people use it. One could use TV to look at Animal Planet and Discovery and learn something, but most use it to watch the dumbest stuff like Ex on the Beach and Hotel Cæsar. In the same way, people use AI to get answers as fast as possible whenever they need it, at demand, instead of using AI modes like study mode using Socratic method to actually repeat it and solidify the connections between our neurons so that one learns and actually knows. You don't get smarter by instant answers. That is not how the human brain works. It needs spaced repetition to actually "cement the knowledge", and one needs to use the knowledge practically. There's a reason why babies take so long to learn to walk and talk. You get smarter by spaced repetition and seeing connections from what you already know and making connections like a gaussian mixture model in reality, and solidifying the connections by having them "fire" serotonin through the synapses when you recall something you know a bit about, which in turn activates nearby neurons by association. This is the treat in reinforcement learning. You don't get this experience when you prompt a question and gets an answer, at least it is so less powerful it does not really work. Human interaction cues like eye contact and smile and voice also helps this process so much since we are "programmed" in our DNA to register this and weight it much more rather than glyphs represented by squares. But there is potential to make us smarter, we just need to learn to use it properly first.